

Where the Tokens Go: Decomposing and Closing the Decode Gap for a 27B Hybrid LLM on Apple Silicon

Anonymous submission

Draft, September 2026

Abstract

Large language models are increasingly run on personal machines, and Apple silicon with unified memory is one of the few consumer platforms that holds a 25–100B-parameter model in a single address space. Single-stream decode on such a machine is usually described as memory-bandwidth bound: every token reads every weight, so throughput should be bandwidth divided by model bytes. Yet the most widely used runner reaches 45% of that bound on Qwen3.8-27B in Q4_K on an M2 Ultra, and 58% of the streaming rate the GPU actually achieves. This paper asks where the rest of the token goes. We decompose a decode token into weight streaming, matvec kernel efficiency, per-dispatch overhead and CPU work between tokens, using command-buffer GPU timestamps so that sub-millisecond kernels are attributed correctly, and we find that at this scale each of the last three costs milliseconds. Working inside DS4, an open-source model-specific C/Metal engine, we add the model and then pull three levers, each behind a runtime switch so it can be ablated: fewer bytes per token, fewer and better-shaped dispatches, and a shorter CPU path between tokens. Decode rises from 29.3 to 33.1 t/s (llama.cpp: 23.8), 81% of the streaming ceiling, with logits that match llama.cpp at an arbitrary decode step. Two results reach beyond this engine. A controlled experiment with the same weights in Q4_0 and Q4_K shows that the block format is worth 7–19% with simple kernels and 1–2% once the large matvecs are fused, because both formats then stream at the same ceiling; MLX reaches that ceiling on its affine 4-bit format without any model-specific fusion. And a 2 ms-per-token cost hiding in the sampler, caused by `isfinite` under `-ffast-math`, shows that the CPU gap between tokens has to be measured on its own. We report three optimizations that did not help and why, and release the code, switches and scripts.

1 Introduction

Open-weight models in the 20–30B range now fit comfortably on a laptop or desktop with 64–192 GB of unified memory, and people run them locally for chat, coding agents and private workloads. On these machines the question is no longer whether a model fits but how fast it decodes, and the standard answer is a roofline argument: a decode step must read every weight once, so tokens per second is bounded by memory bandwidth over model bytes. For Qwen3.8-27B in Q4_K, 15 GB per token on an M2 Ultra with 800 GB/s nominal, that bound is 53 t/s; the fastest kernel on this GPU streams at 670 GB/s, which puts the practical ceiling at 41 t/s. llama.cpp decodes the model at 23.8 t/s (§7). More than half of every token is something other than moving weights.

That half is hard to see, which is why it persists. A profiler that synchronizes at stage boundaries pays about 0.16 ms per boundary on this machine, more than most of the kernels it is trying to measure, so small kernels vanish into the floor. A system trace reports the GPU 96–98% busy for both llama.cpp and our engine, which sounds like the roofline has been reached; it has not, because a kernel that is running is not a kernel that is streaming at full rate. And a 2 ms cost per token

in the CPU sampler, one seventh of the GPU time, is invisible to every GPU-side tool. The first contribution of this paper is a way of measuring that makes each of these visible: per-stage GPU timestamps from the command buffers, a CPU inter-token split, and reference logits at an arbitrary decode step so that every change is checked where decode actually runs.

With that instrument we pull three levers inside DS4, an open-source model-specific engine written in C and Metal, after adding the model to it. The levers are the bytes per token and how fast they move (the quantization format and the matvec inner loop), the number and shape of dispatches per token (fusing a layer’s four side projections into one launch, the SwiGLU gate and up into one kernel, each residual add into the norm that consumes it, and scaling split-KV attention to the context), and the CPU path between tokens. Each lever has a switch, so the final build can reproduce any subset and switching everything off reproduces the original 29.2 t/s exactly. The result is 33.1 t/s, 81% of the ceiling.

Four findings are, we think, more useful than the speedup.

- **At 27B scale, dispatch count is a first-order term.** The original graph issued about 1,070 dispatches per token; at a few microseconds of ramp and drain each, that is as much time as the recurrence, the attention and every norm together. Fusion is worth 3.9 t/s on its own.
- **The matvec limiter is activation traffic, not occupancy.** Splitting reductions across more threads (K-split) did nothing for the wide projections; reading the activation once per four rows instead of two, and reading it as vectors, did. The Q4_K kernels end at 613 GB/s against 670 GB/s for Q8_0, and the difference is dequantization work.
- **Format and fusion are not additive.** With simple kernels the cheaper Q4_0 block format is worth 19% in llama.cpp and 7% in DS4; with fused kernels it is worth 1–2%, because both formats then stream at the same ceiling. MLX, a general framework with no model-specific graph, reaches that ceiling on its affine 4-bit format. Which lever matters depends on how far the kernels already are from the ceiling.
- **The CPU is on the critical path.** The 2 ms sampler cost was an `isfinite` call left as a library function by `-ffast-math`; the first fix was folded away by the same flags and broke NaN handling. Both facts are general to any inference code built that way.

Our contributions are:

1. A measurement method for Metal decode that attributes token time to weight streaming, matvec efficiency, per-dispatch overhead and CPU gaps, with a correctness protocol at arbitrary decode steps (§3, §6).
2. Qwen3.8-27B support in a model-specific engine, and a set of decode optimizations each measured in isolation and each behind a switch (§5, Table 7).
3. A controlled quantization-format experiment on identical weights, in two engines and at two fusion levels (§7.4), and a comparison with MLX across context lengths (§7).
4. Negative results with mechanisms (§8), a profile of a mixture-of-experts model on the same instrument, and the code, switches and scripts.

DS4 (DwarfStar) is by Salvatore Sanfilippo and contributors [21] (<https://github.com/antirez/ds4>); it implements a handful of architectures as hand-written graphs rather than interpreting a

Component	Weights	Q4_K bytes
DeltaNet layer ($\times 48$): qkv, z, α, β , out, FFN	383.3M	215.6 MB
Attention layer ($\times 16$): $q gate, k, v$, out, FFN	372.3M	209.4 MB
Layers total		13.70 GB
Output head, Q8_0 (248320×5120)	1.27B	1.35 GB
Per token		15.05 GB

Table 1: Bytes streamed per decode token for Qwen3.8-27B in pure Q4_K (0.5625 B/weight) with a Q8_0 output head (1.0625 B/weight). The token embedding is read for one row only.

generic GGUF computation graph. Our work adds Qwen3.8-27B, including its Gated DeltaNet and gated-attention kernels, and the decode paths and measurements described here; the branches and experiment scripts are at <https://github.com/parasol-aser/ds4>. §9 argues that the model-specific trade is the right one for the small set of models people actually run locally.

2 Background

2.1 Platform

The M-series SoCs share one LPDDR5 pool between CPU and GPU. Metal compiles kernels at runtime from Metal Shading Language and executes them from command buffers; a compute encoder can hold many dispatches, which execute serially with memory ordering. There is no host-to-device copy, but the CPU and the GPU share bandwidth, and the CPU-side work between two tokens is on the critical path.

All measurements are on a Mac Studio with an M2 Ultra (24 CPU cores, 60 GPU cores, 192 GB LPDDR5, 800 GB/s nominal) running macOS 26.1 with Apple clang 17. Baselines are llama.cpp at commit 6d9c82ea2 (2026-09-09) with its Metal backend [8], and mlx-lm 0.31.3 [17]. All GGUF engines read the same files from the Unsloth release [22]; MLX uses the community 4-bit conversion of the same checkpoint.

2.2 Model

Qwen3.8-27B (GGUF architecture `qwen35`) is a dense hybrid of 64 layers [20]. In three layers out of four the token mixer is Gated DeltaNet [23]: one fused projection produces q, k (16 heads of 128) and v (48 heads of 128) that share a causal depthwise convolution of width 4; each value head keeps a 128×128 recurrent state updated by the delta rule with a scalar per-head decay $\exp(A \cdot \text{softplus}(\alpha + b))$ and a write strength $\sigma(\beta)$, where α and β come from two 48-wide projections; the output is RMS-normalized per head and gated by $\text{SiLU}(z)$ from a 6144-wide projection. Every fourth layer is gated grouped-query attention: 16 query heads of dimension 256 with 4 KV heads, NEOX RoPE on the first 64 dimensions, per-head RMSNorm on q and k , and the output multiplied by a sigmoid gate that is projected alongside q . All layers have a SwiGLU FFN of hidden size 17408; the width is 5120 and the vocabulary has 248,320 entries. Table 1 gives the bytes a decode step reads.

2.3 Roofline

At 15.05 GB per token the nominal bound is 53 t/s. The practical ceiling is what the GPU actually streams: the fastest kernel we observe is the Q8_0 output-head matvec at 670 GB/s (1.35 GB in 2.01 ms), and the fastest Q4_K kernel reaches 613 GB/s. Taking 620 GB/s as achievable gives 24.3 ms per token, or 41 t/s. We use 41 t/s as the ceiling throughout.

3 Measurement method

Per-stage GPU spans. DS4 can close and reopen its command buffer at named stage boundaries inside a layer (projections, convolution and recurrence, attention, output projection, FFN gate/up, FFN down, residual and norm) under an environment switch. We time each stage with the command buffer’s `GPUStartTime` and `GPUEndTime`. Timing the same boundaries with the CPU clock, as the engine’s earlier profiler did, includes a synchronization of about 0.16 ms per boundary, which is larger than the recurrence, the attention or any norm. Isolating a stage in its own command buffer still adds a fixed per-buffer cost of a few microseconds, so the sum over stages exceeds the unperturbed token time by a few percent (30.9 ms against 30.2 ms for the final build); we use the profile for ranking and for deltas, and unperturbed runs for throughput.

System traces. Metal System Trace (`xctrace`) records GPU busy intervals and command-buffer submissions process-wide; from it we take GPU busy fraction, GPU time per token and CPU encode time per token, for DS4 and llama.cpp on the same prompt.

CPU inter-token split. The CLI generation loop reports, under a switch, the wall time per token spent in sampling, in token emission and in the forward call. Neither GPU view exposes this, and it is where the sampler cost of §5.3 was found.

Reference logits. A small program on the llama.cpp API prefills a prompt, greedy-decodes, and writes the full logit vector at a chosen step k ; DS4 writes the logits of its k -th forward under a switch. We report mean and maximum absolute difference over the 248,320 entries and whether the argmax agrees. With greedy decoding on both sides the token histories are identical up to k whenever the argmax agreed at every earlier step, so the comparison exercises the decode kernels, not only prefill.

4 Anatomy of a decodetoken

Table 2 is the per-stage profile of one decode token on the final build with every optimization switched off and on, and Figure 1 groups it. Three observations organize the rest of the paper.

The large matvecs are near, not at, the ceiling. The FFN gate/up pair moves 100.3 MB per layer. As two ggml-style matvecs plus a SwiGLU kernel it streams at 512 GB/s; fused and vectorized (§5) at 613 GB/s. The 5120-output projections (FFN down, attention and DeltaNet output) sit at 390–450 GB/s with the ggml-style kernel. The Q8_0 output head reaches 670 GB/s on the same memory system, so part of the Q4_K deficit is the ALU cost of unpacking its 6-bit sub-block scales and minimums.

Layer	Stage	Optimizations off		On		On, 8.6k ctx ms/token
		μ s/layer	ms/token	μ s/layer	ms/token	
both	FFN gate/up	196.0	12.54	163.6	10.47	10.49
both	FFN down	111.5	7.14	94.6	6.05	6.05
DeltaNet	projections qkv, z, α, β	116.0	5.57	93.6	4.49	4.50
attention	projections $q gate, k, v$	95.5	1.53	81.5	1.30	1.30
DeltaNet	output norm + out projection	45.5	2.19	47.2	2.27	2.28
attention	gate + out projection	42.7	0.68	44.1	0.71	0.71
DeltaNet	conv + recurrence	23.7	1.14	23.6	1.13	1.13
attention	split-KV attention (+pad, reduce)	105.9	1.69	60.5	0.97	3.38
attention	q/k prologue, V store	7.6	0.12	7.4	0.12	0.13
both	residual adds and norms	28.3	1.82	21.7	1.39	1.40
	output head (Q8.0)		2.01		2.01	2.01
total			36.43		30.92	33.39

Table 2: GPU time per decode token by stage, Qwen3.8-27B pure Q4_K, M2 Ultra, 40-token context, from the per-stage profiler on the final build. “Off” disables the six switches of Table 7; the unswitchable single-launch prologue is present in both columns. The last column is the “on” build at an 8,590-token context. Each stage runs in its own command buffer, so totals exceed the unperturbed token times (34.2 and 30.2 ms).

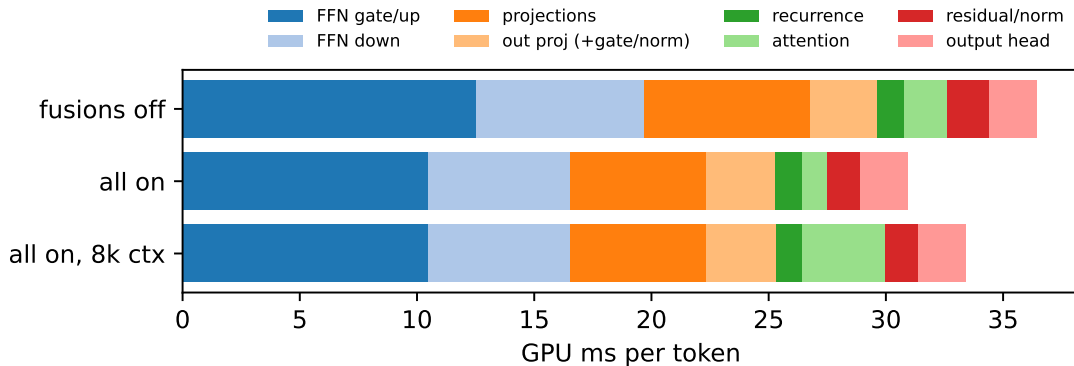


Figure 1: Table 2 grouped by stage type.

Dispatch count matters at this scale. The pre-optimization graph issued about 1,070 dispatches per token (16 per DeltaNet layer, 19 per attention layer, plus embedding and head); at a few microseconds of ramp and drain each this is 3–5 ms, as much as the recurrence, the attention and all norms together. The final graph issues about 620.

GPU busy is not GPU useful. On the mixed-recipe UD-Q4_K_M file, the system trace shows DS4 at 98.3% GPU busy against 95.6% for llama.cpp, with about 1 ms of CPU encode time per token against 4.9 ms. Both engines keep the GPU occupied; most of the gap to the ceiling is inside kernels rather than between them, which is why the work below is mostly about kernel shape and fusion.

5 Optimizations by lever

Every optimization below has an environment switch, so the final build can reproduce any subset. Table 7 and Figure 2 give the single-switch ablation; all switches off reproduces the original 29.2 t/s.

5.1 Bytes: how fast a byte moves

The format fixes the byte count; the kernel decides how close to the streaming rate the bytes move. Two changes to the Q4_K inner loop were worth 9.6% on the fused gate/up kernel (178.6 to 162.9 μ s per layer): staging the 32 activation values a lane needs as eight `float4` loads instead of 32 scalar loads (the activation pointer’s alignment is not provable to the compiler, so it does not vectorize the loads itself), and loading each lane’s two 8-byte nibble runs as `uint2` instead of four `uint16`. Amortizing one activation staging over four rows per simdgroup instead of two helped the 10240- and 12288-output projections in the stage profile and hurt the 5120-output ones, so the row count is chosen per shape; end to end the difference is within noise (Table 7). A K-split variant, which spreads one row’s reduction over the simdgroups of a threadgroup and combines through threadgroup memory, helps only the long-row FFN down projection (17408 inputs: 111 to 94 μ s) and is used only there.

5.2 Dispatches: fewer and better shaped

Multi-weight matvec. A DeltaNet layer’s four side projections (qkv, z, α, β) share one activation and one input width. One kernel with up to four weight segments, dispatched as a single grid whose threadgroup ranges map to segments, writes the outputs back to back into the existing batch buffer; the convolution and recurrence read z, α and β through views. The same kernel issues $q|gate, k$ and v of an attention layer. Four launches become one, and the 48-output α and β heads no longer each pay a dispatch.

Fused gate/up SwiGLU. One kernel streams the gate and up weights against one staged activation and writes $\text{SiLU}(g) \cdot u$; the elementwise SwiGLU pass disappears and the activation is read once per four rows instead of once per two.

Residual add fused into the next norm. Each residual add is fused with the RMSNorm that consumes its result, so a layer’s FFN epilogue produces the next layer’s attention-norm input and the two adds and two norms per layer become two kernels.

Single attention prologue. Per-head RMSNorm plus RoPE for the query heads (to the f32 scratch), the same for the key heads (into the f16 K cache) and the V store are one grid with three kinds of threadgroup instead of three launches.

Context-scaled split-KV. The split-KV decode attention kernel inherited from ggml partitions the keys over 32 workgroups and reduces their partial results in a second kernel. At a 40-token context 30 workgroups are idle and the reduce dominates. Scaling the workgroup count with the context length, and letting a single workgroup write its normalized output directly, cuts the attention stage from 106 to 61 μ s per layer. This required making the shared reduce kernel correct for fewer than 32 workgroups, which it had silently assumed.

Prompt	mean $ \Delta $	max $ \Delta $	argmax
22-token chat prompt	0.012	0.179	agrees
1439-token prompt	0.024	0.214	agrees

Table 3: Logit difference from llama.cpp at decode step 4, pure Q4_K, over 248,320 entries.

File	bytes/weight	wikitext-2 PPL
pure Q4_K, imatrix	0.5625	5.80 ± 0.07
UD-Q4_K.M	0.58	5.85 ± 0.07
Q8_0	1.0625	5.86 ± 0.07
MLX 4-bit affine, group 64	0.5625	5.98
pure Q4_0, imatrix	0.5625	6.03 ± 0.07

Table 4: Perplexity on the first 40 chunks of 2048 tokens of wikitext-2 test, scoring the second half of each chunk (llama-perplexity for the GGUF files; a script with the same chunking for the MLX model, which reports no standard error). Bytes per weight are for the layer tensors. The MLX conversion uses no importance matrix.

5.3 The CPU path between tokens

With sampling at the model card’s defaults (temperature 1.0, top- p 0.95, no top- k), the CLI spent 2.2 ms per token in the sampler against 30.4 ms in the forward call. The cost was not the softmax or the sort but `isfinite()` on each of the 248,320 logits: the engine is compiled with `-ffast-math`, which on this toolchain leaves `isfinite` as a library call and keeps the scan scalar at about 4 ns per element, twice per token. A first replacement, an exponent-bit test on the float value, was folded to “always finite” by the optimizer under the same flags, admitted NaN and $-\infty$ logits, and failed 1,459 unit tests. The working version tests the bits as loaded from memory, which the optimizer cannot classify. With the max scan and the top- p pass then written branch-free and block-wise so they vectorize, a degree-5 fit of 2^x for the tail mass that cannot enter the 512-entry candidate heap, and a lowering-cutoff fallback instead of a full-vocabulary sort, sampling costs 0.5 ms per token. The two lessons are general: profile the inter-token gap on the CPU, and treat fast-math flags as part of the numerical contract of any code that handles non-finite values.

6 Correctness

Every change was checked against llama.cpp logits at decode step 4 on two prompts (Table 3); the values are identical across all optimizations, and prefill logits are bit-identical between the initial and final builds because no prefill kernel changed.

Because both engines read the same file and agree to this precision, the quality of a file is a property of the quantization, not the engine. Table 4 gives wikitext-2 perplexity [16] per file. The three GGUF recipes are indistinguishable within their standard errors, so the 41% decode advantage of pure Q4_K over Q8_0 in Table 6 is free on this metric; the two affine 4-bit variants (MLX, Q4_0) cost 3–4%.

File	correct / 10	cut off by the budget
pure Q4_K	8	3
pure Q4_0	7	3
Q8_0	7	4

Table 5: ds4-eval, first ten built-in questions, thinking on, 2,500-token reply budget, greedy.

Baseline	File	DS4 before	greedy DS4 after	baseline	sampld DS4 after	baseline	prefill DS4 / baseline
llama.cpp	pure Q4_K	29.3	33.1	23.8	32.8	23.8	268 / 266
llama.cpp	pure Q4_0	—	33.6	28.4	33.6	—	289 / 304
llama.cpp	UD-Q4_K_M	27.0	28.0	22.8	27.1	22.8	261 / 260
llama.cpp	Q8_0	22.9	23.5	20.0	22.8	20.1	295 / 303
MLX	4-bit affine, g64	—	—	34.1	—	—	268 / 217

Table 6: Decode t/s on a 22-token chat prompt, 64 tokens, M2 Ultra. “Baseline” is the named engine on the file of that row; the MLX row is compared with DS4 on pure Q4_K. “Before” is the initial DS4 implementation (all switches off). Sampled runs use temperature 1 and top- p 0.95. Prefill is on the 1,439-token prompt. llama.cpp’s sampled and greedy rates coincide because its default sampler applies top- $k = 40$ before top- p , which leaves nothing expensive to do; DS4 follows the model card (no top- k) and scans the full distribution.

6.1 Downstream check

As a coarse check on reasoning tasks we ran DS4’s built-in evaluation harness on its first ten questions (drawn from GPQA Diamond, SuperGPQA and AIME 2025) with thinking enabled, greedy decoding and a 2,500-token reply budget (Table 5). The scores are within one question of each other, which is the resolution of ten items; three to four answers per file were cut off by the budget, so the numbers measure quantization and budget together. We include them for completeness, not as evidence of a difference.

7 Evaluation

7.1 Throughput

Table 6 summarizes decode throughput. On pure Q4_K, 33.1 t/s is 30.2 ms per token: about 26 ms of weight streaming at 560–620 GB/s, 2 ms for the output head, and 2–3 ms of small kernels and dispatch overhead; 81% of the 41 t/s ceiling and 63% of the nominal bound. Against llama.cpp the gain is 39% on pure Q4_K, 23% on the mixed-recipe file (only its Q4_K tensors take the fused paths) and 18% on Q8_0 (only the format-independent fusions apply).

With sampling, DS4 loses 0.3 t/s on Q4_K (the 0.5 ms sampler of §5.3; 2.2 ms before it) and 0.9 on the mixed-recipe file. In the interactive REPL with thinking enabled over 256 generated tokens it reaches 32.2 t/s.

7.2 Ablation

The fused gate/up kernel is the largest single item, followed by the multi-weight projections; the vectorized single-weight matvec and the K-split overlap because both replace the ggml-style kernel

Configuration	decode t/s	Δ
all optimizations on	33.1	
no fused gate/up SwiGLU	31.1	-2.0
no multi-weight projections	31.7	-1.3
no vectorized single-weight matvec	32.0	-1.1
no K-split (FFN down)	32.0	-1.1
fixed 32 split-KV workgroups	32.8	-0.3
no fused residual+norm	32.9	-0.2
multi-weight kernel at 2 rows/simdgroup	33.2	+0.1
all off	29.2	-3.9

Table 7: Single-switch ablation, pure Q4_K, greedy, 22-token prompt, 64 tokens, mean of two runs; run-to-run spread is about ± 0.1 t/s.

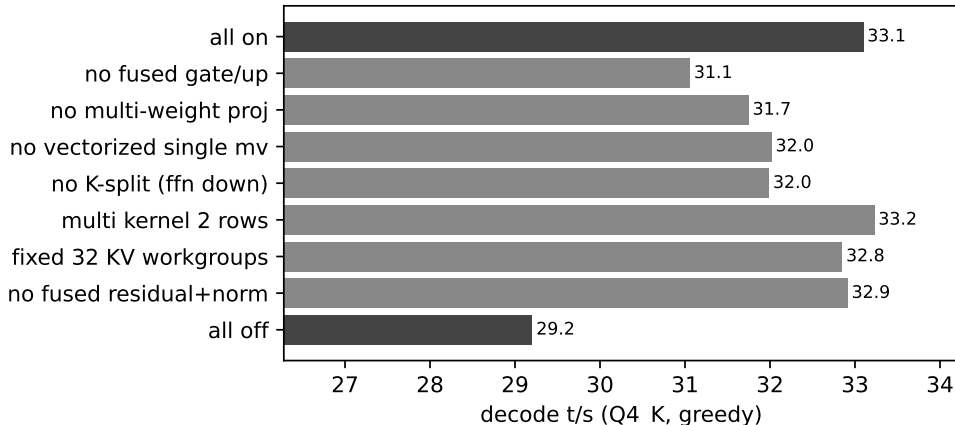


Figure 2: Decode throughput with one optimization removed at a time.

on the FFN down projection. The residual+norm fusion and the context-scaled attention are small at a 40-token context and the sum of the switches (6.0 t/s) exceeds the end-to-end difference (3.9) because several remove the same dispatch overhead.

7.3 Context length and MLX

Figure 3 and Table 8 sweep the prompt from 1.4k to 33k tokens. Decode falls from 32.3 to 26.8 t/s for DS4 and from 23.6 to 20.3 for llama.cpp. The DeltaNet layers cost the same at every length (Table 2); the decline is the sixteen attention layers reading a K/V cache that grows by 64 KiB per position summed over those layers, plus the split-KV reduce (0.97 to 3.38 ms per token from 40 to 8.6k tokens). Prefill throughput is the same in DS4 and llama.cpp within 3% at every length: prefill is a matrix-matrix workload on comparable dequantize-and-multiply kernels, and the model-specific graph does not pay there.

MLX is the informative comparison. Its 4-bit affine format (a 16-bit scale and bias per group of 64, 4.5 bits per weight, the same as Q4_K) decodes at 33.5 t/s at 1.4k context and stays within 2.3 t/s of DS4 at every length, from a general framework whose model code is a few hundred lines of Python over its primitives; its prefill is 13–19% slower. We read the decode parity as a statement

Prompt tokens	decode			prefill		
	DS4	MLX	llama.cpp	DS4	MLX	llama.cpp
1450	32.3	33.5	23.6	268	217	266
4306	29.9	32.2	22.6	272	224	266
8590	31.3	31.0	22.8	268	224	261
17158	29.5	29.9	21.9	258	219	250
32866	26.8	27.5	20.3	239	208	232

Table 8: Throughput (t/s) versus prompt length, greedy, 64 decoded tokens, single runs. DS4 and llama.cpp on pure Q4_K, MLX on its 4-bit affine conversion. The 4k/8k inversion for DS4 is within the ± 1 t/s single-run spread at these lengths.

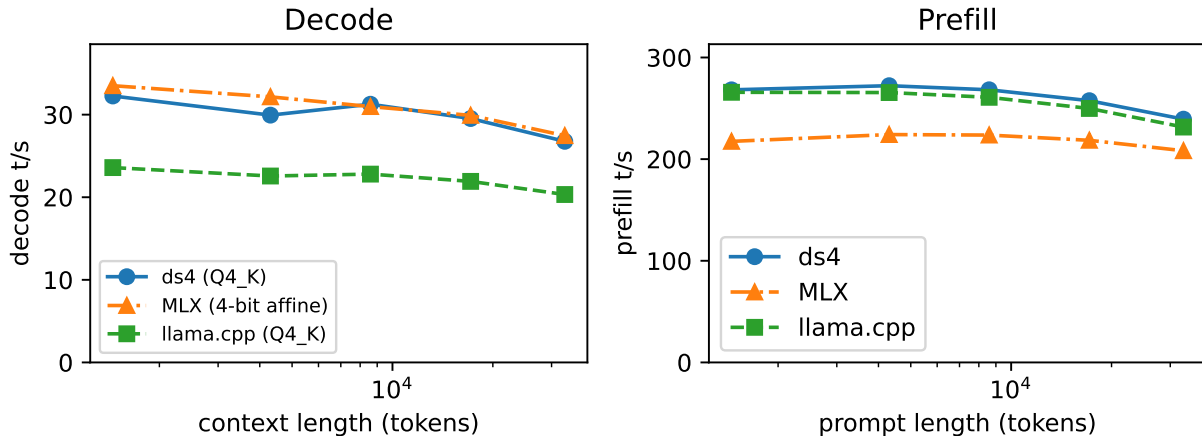


Figure 3: Decode and prefill throughput versus prompt length.

about the format, not the graph: affine dequantization is a multiply-add per weight, and §4 showed that the Q4_K matvecs are held below the Q8_0 streaming rate by dequantization work. §7.4 tests this directly.

7.4 The format lever: Q4_0 against Q4_K

To separate the quantization format from everything else we built a pure Q4_0 file from the same Q8_0 source with the same importance matrix and the same Q8_0 output head as the pure Q4_K file: identical bit budget (16.0 GB on disk), different block layout. Q4_0 stores one 16-bit scale per 32 weights and dequantizes with one multiply-add per weight; Q4_K packs 6-bit sub-block scales and minimums into a 256-weight super-block that must be unpacked. We ported the ggml-style Q4_0 matvec into DS4 and then the fused gate/up and multi-weight kernels; Table 9 gives the result at each kernel level in both engines.

With simple kernels the format alone is worth 19% in llama.cpp and 7% in DS4: the Q4_0 gate/up pair streams at 171 μ s per layer against 196 for Q4_K. With the fused kernels the two formats land at 33.6 and 33.1 t/s. The stage profile explains why the gains do not compose: the fused gate/up kernel takes 166 μ s in Q4_0 and 164 in Q4_K, both about 610 GB/s, so once the largest projection is fused the cheaper dequantization has nothing left to buy there. Q4_0 still gains on the narrower projections (84 against 94 μ s for the DeltaNet projections, 75 against 82 for

Engine	Kernels	Q4_K	Q4.0
llama.cpp	stock	23.8	28.4
DS4	per-format ggml-style matvecs, generic fusions	29.5	31.7
DS4	+ fused gate/up and multi-weight kernels	33.1	33.6
wikitext-2 PPL		5.80 ± 0.07	6.03 ± 0.07

Table 9: Decode t/s for the same weights in Q4_K and Q4.0 (both pure, same imatrix, Q8_0 head), greedy, 22-token prompt. “Generic fusions” are the residual+norm fusion, the single prologue and the context-scaled attention, which do not depend on the format. Q4.0 logits agree with llama.cpp to a mean $|\Delta|$ of 0.0009 at decode step 4.

Profiler stage (43 layers)	$\mu\text{s}/\text{layer}$	ms/token
routed_moe	201	8.65
q_path	164	7.06
attn_inv_rope	134	5.75
attn_output	117	5.05
router	51	2.18
kv_path	32	1.36
attn_hc_pre, ffn_hc_pre (hyper-connection mixes)	26 each	2.27
shared_down	23	1.00
compressor and indexer updates (amortized)		0.51
total		33.8

Table 10: Per-stage GPU time for DeepSeek V4 Flash decode on DS4, 40-token context, using the engine’s stage labels. A stage whose kernels are encoded in a neighbour’s command buffer is attributed to that neighbour, which is why some labels report zero and are omitted.

attention) and loses on the FFN down projection (103 against 95), which has a K-split kernel only for Q4_K. The format lever therefore matters in proportion to how far the kernels are from the streaming ceiling, and MLX’s 34.1 t/s on the affine format sits at that same ceiling.

7.5 A second family: mixture of experts

DS4 was written for DeepSeek V4 Flash, a mixture-of-experts model, and its decode there is shaped by a different lever: only the routed experts of a token are read, so bytes per token depend on routing, and the engine gathers expert weights from a combined buffer and fuses the router with the selection. On the 86 GB mixed-recipe file (IQ2_XXS routed experts, Q8_0 attention projections, shared experts and head), DS4 decodes at 30.3 t/s against 25.2 for llama.cpp on the same file (greedy, short prompt, 64 tokens). Table 10 is the same profiler on this model. The routed experts, the query path and the attention output projection account for two thirds of the token, and no single projection dominates the way the FFN does in the dense model; the decomposition applies per family with a different dominant term, and the specific kernels do not transfer.

8 What did not work

K-split on wide matrices. Splitting the reduction across the simdgroups of a threadgroup multiplies the thread count for a fixed number of output rows. It helped only the long-row FFN

down projection and was neutral or slightly worse on the 10240- and 12288-output projections; occupancy was not their limiter, the activation re-read per row was, and rows per simdgroup was the parameter that moved.

Output gate fused into the attention reduce. A copy of the split-KV reduce that multiplies by $\sigma(\text{gate})$ before its single store was consistently 20 μs per layer slower than the shared reduce followed by a trivial elementwise gate kernel, even with the gate arithmetic removed. The copy specializes its loops on function constants where the shared kernel takes them as runtime arguments; we did not confirm that this is the cause, and dropped the fusion.

Convolution folded into the recurrence. Each of the 1,536 recurrence threadgroups would recompute the causal convolution for its head’s 256 q/k channels, a 32-fold redundant read of the convolution state. The estimated cache traffic exceeded the one dispatch it would save.

9 Discussion

Model-specific against generic. DS4 implements each supported architecture as a hand-written graph. The cost is that each new family is a port; the benefit is that every fusion above is a local change with a measured effect and the per-token dispatch list is known and short. A generic graph runner can obtain the same fusions from a pass over its graph, and llama.cpp and MLX both fuse some operators, but the two decode-specific fusions that mattered most here, batching a layer’s side projections into one launch and scaling the split-KV work to the context, are decisions about the model and the session rather than about the graph. The MLX result shows that a cheaper format buys the same as our kernel work at this scale; both together would be the next step, and the engine now has the Q4_0 kernels for it.

What is left. The remaining 20% to the ceiling is split between the latency-bound recurrence, the attention path, and per-dispatch ramp; §7.4 shows that dequantization is no longer one of the terms once the large matvecs are fused. Beyond that the model ships a multi-token prediction head, and speculative decoding with it would verify two tokens per weight pass; we have not implemented it.

10 Related work

Local inference runtimes. llama.cpp and ggml [8, 7] are the reference for consumer-hardware inference; their Q4_K and Q4_0 matvecs are the starting points of the kernels we compare against, and their split-KV decode attention is the one we adapt. MLX [10] is Apple’s array framework; mlx-lm [17] implements the Gated DeltaNet families on it.

Roofline and phase analyses. Memory-bound decode and the prefill/decode split are the basis of FlashAttention and Flash-Decoding [3, 4] and of disaggregated serving [18, 25]. Those analyses target discrete GPUs; the unified-memory setting puts the CPU on the inter-token critical path and makes per-dispatch cost visible at 27B scale.

Hybrid linear-attention models. Gated DeltaNet [23] and the Qwen3-Next and Qwen3.5 families [19, 20] interleave linear-attention layers with a minority of softmax-attention layers; the reference CUDA kernels [24] use chunked and recurrent forms that do not transfer to Metal.

Quantization formats. Weight-only quantization methods [6, 15], the k-quant block formats [11], importance-matrix calibration [12] and mixed-recipe files [22] trade bytes for dequantization work; §7.4 measures that trade on Apple GPUs.

Speculative and multi-token decoding. Verifying several tokens per weight pass [13, 2, 1, 14] is the standard route past the streaming bound; the model used here ships a multi-token prediction head [9, 5].

11 Limitations

All results are from one machine, an M2 Ultra with a 60-core GPU. The achieved bandwidth fractions and the balance between levers will differ on other generations; in particular the M5 family adds GPU matrix units that change the prefill regime, and a device sweep is the obvious extension. The decomposition is for one dense hybrid family; the MoE result is a throughput and a profile, not an ablation. The per-stage profiler perturbs what it measures by isolating stages, so we use it for rankings and deltas only. The sampler finding depends on compiler flags and may not reproduce under `-fno-finite-math-only`. The downstream check has ten items.

12 Conclusion

A decode token on Apple silicon is not simply bandwidth bound at the 27B scale: matvec kernel shape, dispatch count and CPU work between tokens each cost milliseconds, and each needs its own measurement. Taking GPU timestamps per stage and timing the CPU gap turned each optimization into a targeted change with a known payoff, the switches make the account auditable, and the controlled format experiment shows where one lever stops and another starts.

References

- [1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.
- [3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, 2023.
- [5] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.
- [6] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

- [7] Georgi Gerganov and contributors. ggml: Tensor library for machine learning. <https://github.com/ggml-org/ggml>, 2023.
- [8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- [9] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning (ICML)*, 2024.
- [10] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [11] Kawrakow. k-quants: 2–6 bit quantization with per-block scales (llama.cpp pull request 1684). <https://github.com/ggml-org/llama.cpp/pull/1684>, 2023.
- [12] Kawrakow. Importance matrix calculation for quantization (llama.cpp pull request 4861). <https://github.com/ggml-org/llama.cpp/pull/4861>, 2024.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- [14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
- [15] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- [17] MLX contributors. mlx-lm: LLM inference and fine-tuning with MLX. <https://github.com/ml-explore/mlx-lm>, 2024.
- [18] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.
- [19] Qwen Team. Qwen3-next: Towards ultimate training and inference efficiency. <https://qwen.ai/blog?id=qwen3-next>, 2025.
- [20] Qwen Team. Qwen3.5 and qwen3.8 model series. <https://huggingface.co/Qwen>, 2026.
- [21] Salvatore Sanfilippo and contributors. ds4 (DwarfStar): a model-specific LLM inference engine in C and Metal. <https://github.com/antirez/ds4>, 2026.
- [22] Unsloth. Unsloth dynamic 2.0 GGUFs. <https://docs.unsloth.ai/basics/unsloth-dynamic-2.0-ggufs>, 2025.
- [23] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

- [24] Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, 2024.
- [25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

A Reproduction

The code is our fork of the DS4 repository [21] at <https://github.com/parasol-aser/ds4>. Three branches build on each other: **general-decode-fixes** (sampler and diagnostics), **qwen38-support** (the model and its decode paths) and **paper-artifact** (this document and the scripts under **paper/scripts/**). Everything below was run on macOS 26.1 with Xcode’s command-line tools; a llama.cpp checkout is needed for the baselines, the reference logits and the quantization.

1. Build

```
git clone https://github.com/parasol-aser/ds4 && cd ds4 && git checkout paper-artifact
make                                     # ds4, ds4-server, ds4-agent, ds4-eval
git clone https://github.com/ggml-org/llama.cpp ~/github/llama.cpp
cd ~/github/llama.cpp && git checkout 6d9c82ea2
cmake -B build && cmake --build build -j
pip3 install -U mlx-lm                  # 0.31.3 or later for the qwen3_5 architecture
```

2. Models

```
./download_model.sh qwen38-q8           # Qwen3.8-27B-Q8_0.gguf
./download_model.sh qwen38-q4km         # Qwen3.8-27B-UD-Q4_K_M.gguf
./download_model.sh ds4f-q2            # DeepSeek V4 Flash mixed-recipe file
# importance matrix from the same Unsloth release, then the two pure files:
cd gguf
curl -LO https://huggingface.co/unsloth/Qwen3.8-27B-GGUF/resolve/main/imatrix_unsloth.gguf
L=~/github/llama.cpp/build/bin
$L/llama-quantize --allow-requantize --pure --imatrix imatrix_unsloth.gguf \
  --output-tensor-type q8_0 --token-embedding-type q4_k \
  Qwen3.8-27B-Q8_0.gguf Qwen3.8-27B-Q4_K.gguf Q4_K_S 8
$L/llama-quantize --allow-requantize --pure --imatrix imatrix_unsloth.gguf \
  --output-tensor-type q8_0 --token-embedding-type q4_0 \
  Qwen3.8-27B-Q8_0.gguf Qwen3.8-27B-Q4_0.gguf Q4_0 8
```

The DeepSeek file we measured is the earlier build of the ds4f-q2 recipe (same name without the -0731 suffix).

3. Reference logits

```
cd paper/scripts && mkdir -p out
c++ -std=c++17 -O2 -I ~/github/llama.cpp/include -I ~/github/llama.cpp/ggml/include \
  logits_dump.cpp -L ~/github/llama.cpp/build/bin -llama -lggml -lggml-base \
  -Wl,-rpath,$HOME/github/llama.cpp/build/bin -o out/logits_dump
# reference: prefill, greedy, dump the logits of step 4
```

```

out/logits_dump ../../gguf/Qwen3.8-27B-Q4_K.gguf "$PROMPT" 8 5 4 out/ref_s4.bin
# ds4: same prompt, dump the logits of its 4th forward
DS4_GLM_LOGIT_DUMP=out/ds4_s4.bin DS4_QWEN_LOGIT_DUMP_STEP=4 \
  ../../ds4 -m ../../gguf/Qwen3.8-27B-Q4_K.gguf -p "$PROMPT" -n 8 --temp 0 -c 4096
python3 cmp.py out/ds4_s4.bin out/ref_s4.bin      # mean/max |diff|, argmax

```

\$PROMPT is the ChatML prompt with an empty thinking block, as in `run_all.sh`.

4. Experiments

```

cd paper/scripts
curl -sL -o out/wikitext-2-raw-v1.zip \
  https://huggingface.co/datasets/ggml-org/ci/resolve/main/wikitext-2-raw-v1.zip
unzip -qo out/wikitext-2-raw-v1.zip -d out
python3 make_prompts.py      # out/prompt_{1k,4k,8k,16k,32k}.txt
./run_all.sh                 # ~30 min: sweeps, ablation, profiles, PPL, MoE
./run_q40.sh                 # ~15 min: pure Q4_0 experiment
./run_mlx.sh                 # download (15 GB) + ~20 min
./run_eval.sh                # ~40 min: downstream check
python3 analyze.py           # tables; python3 figures.py -> ../figs/

```

The GPU must be otherwise idle: DS4 refuses to start while another DS4 process holds `/tmp/ds4.lock`, and timing is not meaningful with a second model process on the GPU.

5. Switches and profilers

```

DS4_METAL_DECODE_STAGE_PROFILE=1 DS4_METAL_CB_TIMES=1 # per-stage GPU spans
DS4_TOKEN_TIMING=1                # CLI: sample / emit / eval per token
DS4_GLM_LOGIT_DUMP=f DS4_QWEN_LOGIT_DUMP_STEP=k      # logits of the k-th forward
DS4_QWEN_DISABLE_FUSED_SWIGLU=1  # fused gate/up -> two matvecs + SwiGLU
DS4_QWEN_DISABLE_MULTI_PROJ=1    # multi-weight proj -> one launch each
DS4_QWEN_DISABLE_FAST_MV=1       # vectorized single-weight mv -> ggml-style
DS4_METAL_Q4K_KSPLIT_NSQ=0       # K-split (FFN down) off
DS4_METAL_QWEN_FA_NWG=32         # fixed 32 split-KV workgroups
DS4_QWEN_DISABLE_FUSED_NORM=1    # residual+norm fusion off
DS4_METAL_Q4K_MULTI_NRO=2        # multi-weight kernel rows per simdgroup

```

Stage-profile logs are aggregated with `stage_agg.py`. The Metal System Trace captures of §4 were recorded and exported with:

```

xcrun xctrace record --template 'Metal System Trace' --time-limit 120s \
  --output ds4.trace --launch -- ./ds4 -m gguf/Qwen3.8-27B-UD-Q4_K_M.gguf -p "$PROMPT" -n 32 --temp
  0
xcrun xctrace export --input ds4.trace \
  --xpath '/trace-toc/run[@number="1"]/data/table[@schema="metal-gpu-intervals"]'
xcrun xctrace export --input ds4.trace \
  --xpath '/trace-toc/run[@number="1"]/data/table[@schema="metal-application-command-buffer-
  submissions"]'

```

GPU busy fraction and per-token GPU time come from the union of the GPU intervals in the decode window; CPU encode time per token from the submission table's encoder durations.